Design and Implementation of an RFSoC-Based Digital Transceiver Using Hardware–Software Co-Design

Name: Mohit Gupta
Institute: Indraprastha Institute of Information Technology Delhi (IIIT-Delhi)
Department: Electronics and Communication Engineering
Platform: AMD/Xilinx Zynq UltraScale+ RFSoC 4x2
Software: Vivado, PYNQ, Python

ABSTRACT

Software Defined Radio (SDR) has become one of the most important technologies in modern wireless communication systems because it enables signal processing to be performed primarily through software while utilizing configurable hardware resources. Traditional SDR systems require multiple discrete components such as high-speed Analog-to-Digital Converters (ADCs), Digital-to-Analog Converters (DACs), Field Programmable Gate Arrays (FPGAs), processors, and RF front-end circuitry. These components increase system complexity, power consumption, latency, and overall design cost.

The AMD/Xilinx Zynq UltraScale+ RFSoC addresses these limitations by integrating multi-core ARM processors, programmable FPGA logic, and multi-gigahertz RF Data Converters on a single chip. Such integration enables efficient hardware-software co-design for applications requiring high-speed data acquisition and real-time signal processing.

The objective of this project is to design and implement a complete digital transmit–receive chain on the RFSoC platform. A baseband sinusoidal waveform is generated in software using Python and transferred to the FPGA through AXI Direct Memory Access (DMA). The integrated DAC converts the digital waveform into an analog RF signal after digital upconversion. The transmitted signal is then captured using the integrated ADC, digitally downconverted, transferred back to the processing system, and analyzed using Fast Fourier Transform (FFT) techniques.

The project demonstrates the interaction between the ARM Processing System (PS), the Programmable Logic (PL), AXI communication protocols, DMA engines, RF Data Converter IP, and Python-based software control through the PYNQ framework. Experimental results verify the correctness of the implemented transmitter–receiver chain through both time-domain and frequency-domain analysis.

INTRODUCTION

1.1 Background

Wireless communication systems have experienced tremendous growth over the last two decades. Technologies such as Wi-Fi, LTE, 5G, satellite communication, radar, Internet of

Things (IoT), and software-defined radio require the processing of large volumes of high-speed digital data while maintaining strict timing constraints.

Conventional communication systems often rely on separate integrated circuits for digital processing, RF conversion, memory, and control. These discrete architectures increase latency, consume more power, and require complex PCB routing between devices.

To overcome these challenges, modern System-on-Chip (SoC) platforms integrate multiple functional blocks into a single device. Among these, the AMD/Xilinx Zynq UltraScale+ RFSoC represents one of the most advanced platforms for wireless communication research. The RFSoC combines:

- Multi-core ARM Cortex-A53 processors
- Programmable FPGA fabric
- Multi-gigahertz ADCs
- Multi-gigahertz DACs
- High-speed memory interfaces
- Digital signal processing resources

into a single integrated chip.

1.2 Motivation

Modern wireless systems continuously demand higher bandwidths, lower latency, and increased computational performance. Applications such as:

- 5G Base Stations
- Massive MIMO
- Software Defined Radio
- Radar Systems
- Electronic Warfare
- Satellite Communication

all require processing of RF signals at extremely high sampling rates.

Traditional FPGA platforms require external ADC and DAC hardware connected through high-speed interfaces. These external devices increase system complexity and require careful PCB design.

The RFSoC eliminates this requirement by integrating RF Data Converters directly within the FPGA architecture.

This project was motivated by the need to understand:

- Hardware-software co-design
- RF signal generation
- RF signal reception
- FPGA-based streaming architectures
- AXI communication protocols
- High-speed DMA transfers
- RF Data Converter configuration

1.3 Problem Statement

Design and implement a complete digital transmitter and receiver system on the AMD Zynq UltraScale+ RFSoC using the integrated RF Data Converter (RFDC), AXI DMA, FPGA programmable logic, and ARM processing system. The system should generate a digital baseband signal, transmit it through the DAC, receive it through the ADC, and verify successful transmission using frequency-domain analysis.

1.4 Expected Outcome

At the completion of this project, the following outcomes are expected:

- Successful generation of a digital baseband signal.
- Correct RF transmission using the integrated DAC.
- Successful reception using the integrated ADC.
- Correct operation of the DMA engines.
- Proper interaction between the Processing System and Programmable Logic.
- Accurate frequency estimation using FFT.
- Demonstration of hardware-software co-design on the RFSoC platform

2 RFSoC Platform

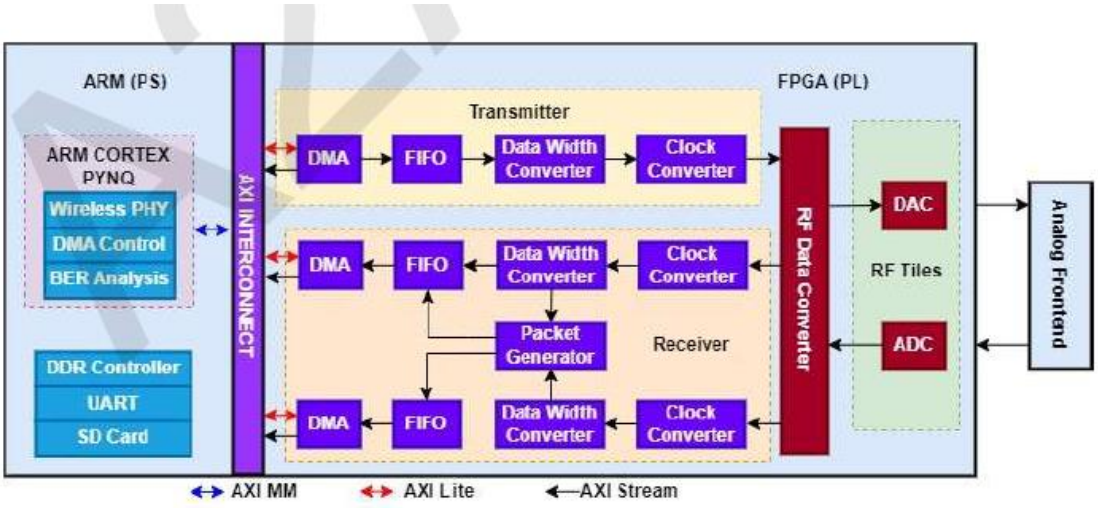
The AMD/Xilinx Zynq UltraScale+ RFSoC is an advanced System-on-Chip (SoC) that combines a multi-core ARM processor, FPGA fabric, and high-speed RF Data Converters within a single silicon device.

Unlike conventional FPGA boards that require external ADCs and DACs, the RFSoC integrates these converters directly on-chip, significantly reducing latency, board complexity, and power consumption.

The RFSoC mainly consists of three major subsystems:

1. Processing System (PS)
2. Programmable Logic (PL)

3. RF Data Converter (RFDC)



2.1Processing System (PS)

The Processing System consists of a Quad-Core ARM Cortex-A53 processor responsible for executing software applications and controlling the hardware.

In this project, the Processing System performs the following tasks:

- Generates the digital sine wave.
- Configures RF Data Converter parameters.
- Initializes RF clocks.
- Controls DMA transfers.
- Starts and stops the transmitter.
- Collects received samples.
- Computes FFT.
- Displays time-domain and frequency-domain plots.

Since these tasks are not latency-critical, they are executed in software using Python through the PYNQ framework.

2.2 Programmable Logic (PL)

The Programmable Logic is the FPGA portion of the RFSoC.

Unlike the ARM processor, which executes instructions sequentially, FPGA logic executes operations in parallel, making it suitable for high-speed digital signal processing.

The Programmable Logic contains:

- AXI DMA
- Packet Generator
- RFDC Interface
- AXI Interconnect
- User-defined Hardware IPs

The FPGA transfers streaming data between the RF Data Converter and the Processing System at multi-gigabit speeds.

2.3 RF Data Converter (RFDC)

The RF Data Converter (RFDC) is one of the distinguishing features of the RFSoC platform.

It integrates:

- Multi-GSPS ADCs
- Multi-GSPS DACs
- Digital Up Converter (DUC)
- Digital Down Converter (DDC)
- Numerically Controlled Oscillator (NCO)
- Digital Mixers
- Interpolation Filters
- Decimation Filters

Instead of requiring external RF hardware, these functions are available directly inside the chip.

The RFDC supports high-speed wireless communication applications including:

- 5G
- Radar
- Satellite communication
- Electronic warfare

2.4 Analog-to-Digital Converter (ADC)

The Analog-to-Digital Converter converts a continuous-time analog signal into discrete digital samples.

The conversion process consists of three major stages:

Sampling

The analog waveform is sampled at regular intervals.

Quantization

Each sampled value is approximated to the nearest digital level.

Encoding

The quantized value is represented as a binary number.

The relationship between analog and digital signals is illustrated below.

According to the Nyquist Sampling Theorem,

$$F_s \geq 2F_{max}$$

where

$F_s$is the sampling frequency.

$F_{max}$is the maximum frequency contained in the signal.

Failure to satisfy this condition results in aliasing.

2.5 Digital-to-Analog Converter (DAC)

The Digital-to-Analog Converter performs the reverse operation.

It converts digital samples into a continuous analog waveform.

The DAC receives digital samples from the FPGA through the RF Data Converter and reconstructs the analog RF signal.

The quality of reconstruction depends upon:

- Sampling frequency
- Quantization resolution
- Reconstruction filtering

In this project, the DAC operates at

$$4.9152\ GSPS$$

allowing direct generation of RF signals.

2.6 Digital Up Conversion (DUC)

The generated signal in software is only a **20 MHz baseband tone**.

Wireless communication systems require transmission at much higher carrier frequencies.

Therefore, the RFSoC performs Digital Up Conversion.

The DUC consists of

- Interpolation
- Digital Filtering
- Frequency Translation

The frequency translation is achieved using digital mixing.

Mathematically,

$$x_{RF}(t) = x_{BB}(t)e^{j2\pi f_c t}$$

where

$$x_{BB}(t)$$

is the baseband signal

and

$$f_c$$

is the carrier frequency.

In this project,

$$f_c = 300\ MHz$$

Thus,

the RFDC shifts the baseband signal to a 300 MHz carrier before transmission.

2.7 Digital Down Conversion (DDC)

At the receiver,

the incoming RF signal is translated back to baseband.

The RFDC performs

- Digital Mixing
- Low-pass Filtering
- Decimation

The mathematical expression is

$$x_{BB}(t) = x_{RF}(t)e^{-j2\pi f_c t}$$

where

the negative frequency shifts the received carrier back to zero frequency.

In the Python code,

adc_block.MixerSettings['Freq']=-300

implements this downconversion.

2.8 Interpolation

Interpolation increases the sampling rate before transmission.

If

$$L$$

is the interpolation factor,

then

$$F_{DAC} = L \times F_{BB}$$

In this project,

$$L = 16$$

Therefore,

the effective DAC sampling rate becomes

$$4.9152\ GSPS$$

Interpolation provides

- smoother waveform reconstruction
- improved spectral characteristics
- higher time resolution

2.9 Decimation

Decimation reduces the sampling rate after reception.

If

$$M$$

is the decimation factor,

$$F_{BB} = \frac{F_{ADC}}{M}$$

where

$$M = 16$$

This reduces computational complexity while preserving the information bandwidth.

2.10 IQ Representation

Instead of representing signals as purely real values,

modern communication systems represent signals using complex numbers.

A complex signal is expressed as

$$x(t) = I(t) + jQ(t)$$

where

$$I(t)$$

is the In-phase component,

and

$$Q(t)$$

is the Quadrature component.

The RFSoC expects data in an interleaved format:

I0

Q0

I1

Q1

I2

Q2

...

Although the transmitted signal in this project is purely real (so $Q = 0$), the interleaved format supports future implementation of modulation schemes such as QPSK, 16-QAM, and OFDM.

2.11 AXI Communication Protocol

The ARM processor communicates with the FPGA using the Advanced eXtensible Interface (AXI).

Three AXI protocols are used in this project.

AXI Memory Mapped (AXI-MM)

Used for high-speed memory access and DMA transactions.

AXI-Lite

Used for configuring hardware registers and controlling IP cores.

AXI-Stream

Used for continuous streaming of high-speed sample data without address overhead.

2.12Direct Memory Access (DMA)

Direct Memory Access enables data transfer between memory and hardware peripherals without CPU intervention.

Instead of the processor copying each sample individually, the DMA engine transfers large blocks of data directly between DDR memory and the FPGA.

Advantages include:

- Higher throughput
- Lower CPU utilization
- Reduced latency
- Efficient streaming

2.13 Fast Fourier Transform (FFT)

The Fast Fourier Transform converts a time-domain signal into its frequency-domain representation.

For an N-point sequence $x[n]$, the Discrete Fourier Transform (DFT) is defined as

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j\frac{2\pi kn}{N}}$$

The FFT is an optimized algorithm for computing the DFT with a computational complexity of

$$O(N\log N)$$

instead of

$$O(N^2)$$

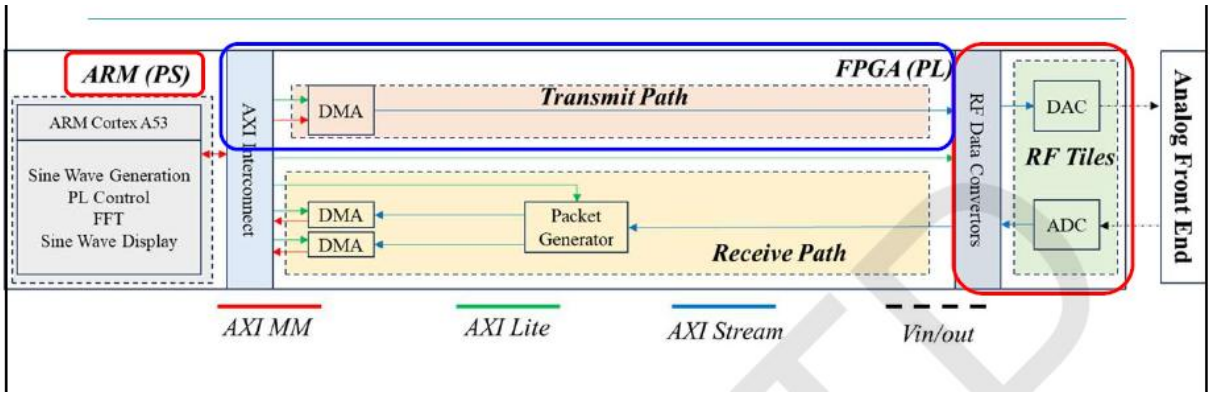
In this project, the FFT is used to verify that the received signal contains the expected 20 MHz baseband tone after digital downconversion.

The magnitude spectrum is computed as

$$20\log_{10} | X(f) |$$

and plotted to identify the dominant frequency component

3. SYSTEM ARCHITECTURE AND HARDWARE DESIGN.



3.1 Processing System (PS)

The Processing System is based on a Quad-Core ARM Cortex-A53 processor running the PYNQ Linux operating system.

The PS acts as the control unit of the entire system.

Its responsibilities include:

- Loading the FPGA bitstream.
- Configuring RF clocks.
- Initializing the RF Data Converter.
- Generating the digital waveform.
- Allocating DMA buffers.
- Starting DMA transfers.
- Receiving captured samples.
- Performing FFT analysis.
- Displaying the results.

Unlike the FPGA, which processes data in parallel, the ARM processor performs sequential execution of software tasks.

The software implementation is written in Python using the PYNQ framework, allowing direct control of hardware IP cores through software APIs.

3.2 Programmable Logic (PL)

The Programmable Logic consists of the FPGA fabric implemented in the Vivado block design.

The FPGA is responsible for real-time processing and high-speed data movement between the Processing System and the RF Data Converter.

The PL contains the following major Intellectual Property (IP) cores:

- AXI DMA (Transmit)
- AXI DMA (Receive I)
- AXI DMA (Receive Q)
- Packet Generator
- AXI Interconnect
- RF Data Converter Interface

The FPGA operates independently of the processor, enabling deterministic and high-throughput data transfer.

3.3 RF Data Converter (RFDC)

The RF Data Converter is the central component of the RFSoC.

It integrates multiple high-speed ADCs and DACs together with digital signal processing blocks.

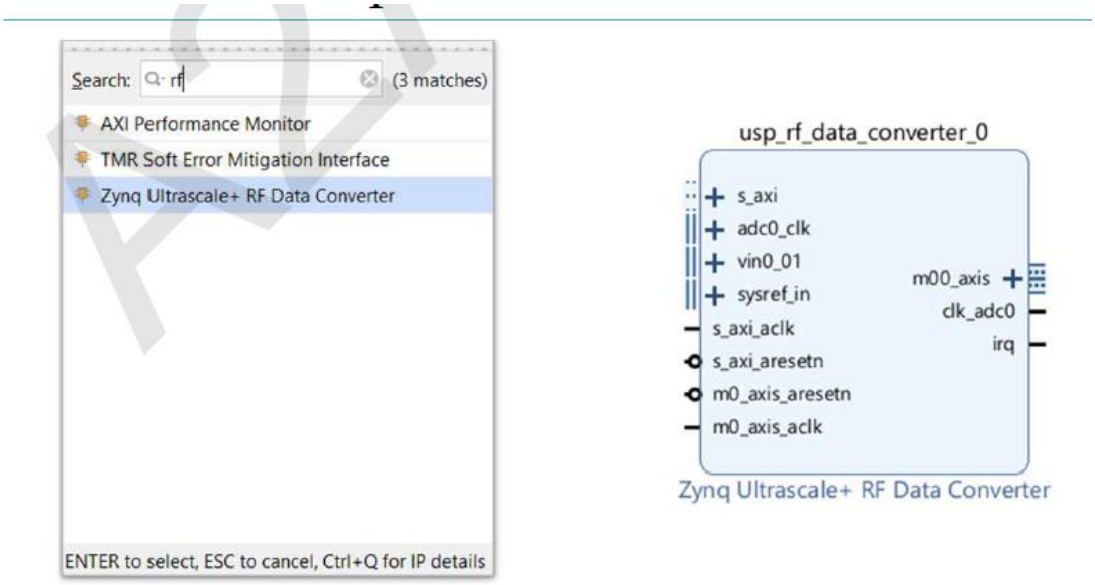
In this project:

- DAC Tile 0 is used for transmission.
- ADC Tile 2 is used for reception.

Each tile consists of:

- Phase Locked Loop (PLL)
- Numerically Controlled Oscillator (NCO)
- Digital Mixer
- Interpolation Filters
- Decimation Filters
- FIFO Buffers

The RFDC eliminates the need for external high-speed RF converters.



3.4 RF Clock Configuration

Accurate clock generation is essential for reliable operation of high-speed RF systems.

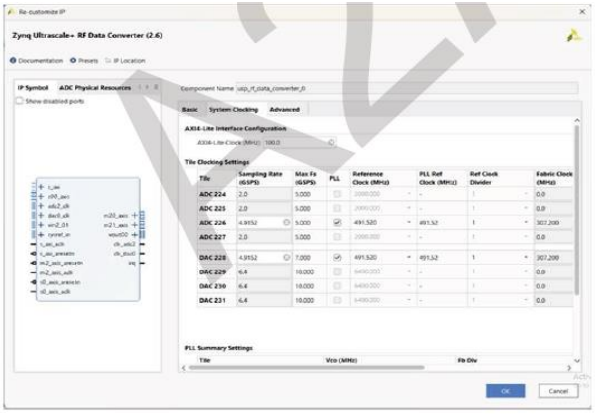
The RFSoC board contains two programmable clock generators:

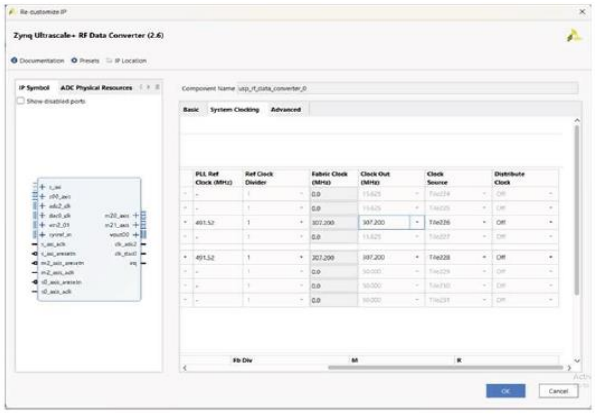
- LMK Clock Generator
- LMX Frequency Synthesizer

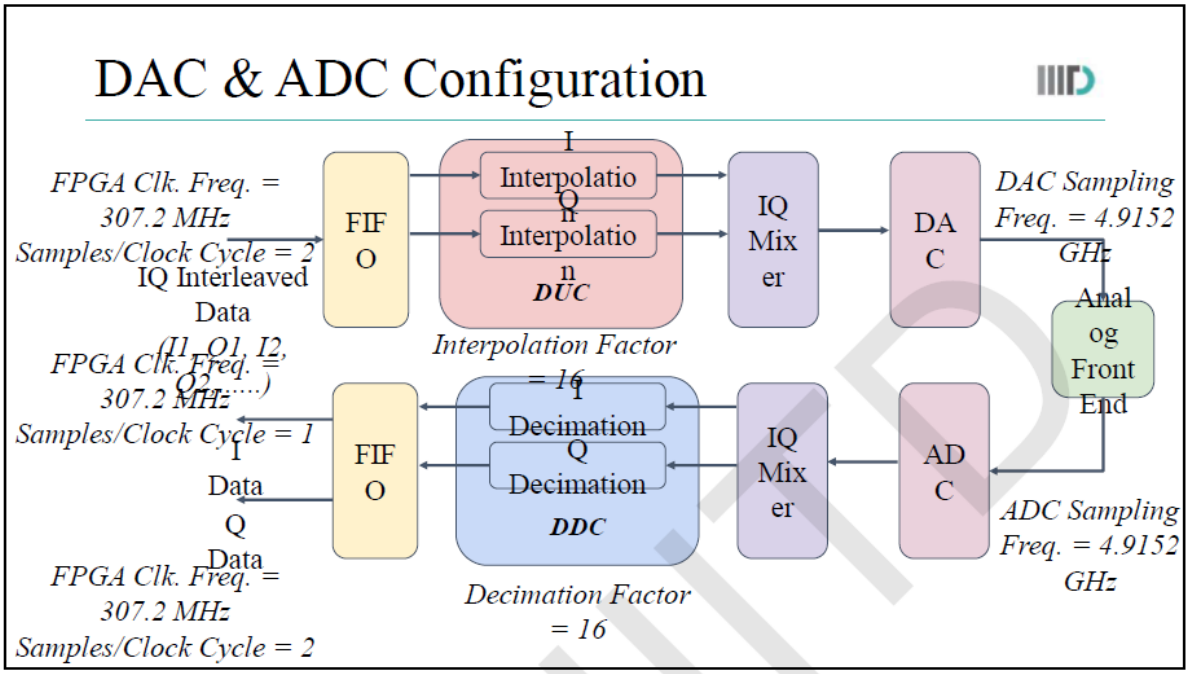
The project configures:

Parameter	Value
LMK Frequency	245.76 MHz
LMX Frequency	491.52 MHz
ADC Sampling Rate	4915.2 MSPS
DAC Sampling Rate	4915.2 MSPS

The clock generators provide synchronized sampling clocks for both the ADC and DAC, ensuring coherent transmission and reception.







3.5 DAC Configuration

The DAC is responsible for converting digital baseband samples into analog RF signals.

The following configuration is used:

Parameter	Value
DAC Tile	0
DAC Block	0
Sampling Rate	4915.2 MSPS
Interpolation Factor	16
Nyquist Zone	1
Mixer Frequency	+300 MHz

The software first configures the DAC PLL, then enables the FIFO, configures the digital mixer, and finally starts the DAC tile.

The interpolation filter increases the effective output sampling rate, allowing smooth waveform reconstruction.

3.6 ADC Configuration

The ADC converts the received analog RF waveform into digital samples.

The following parameters are configured:

Parameter	Value
ADC Tile	2
ADC Block	0
Sampling Rate	4915.2 MSPS
Decimation Factor	16
Nyquist Zone	1
Mixer Frequency	-300 MHz

The negative mixer frequency performs digital downconversion, translating the received RF signal back to baseband before further processing.

3.7 AXI Communication Interfaces

Communication between the Processing System and Programmable Logic is performed using the Advanced eXtensible Interface (AXI).

Three AXI protocols are used.

AXI Memory Mapped (AXI-MM)

Used for:

- DMA memory transactions
- Reading and writing registers
- High-bandwidth memory access

AXI Lite

Used for:

- Packet Generator configuration
- RFDC register configuration
- Peripheral control

AXI Stream

Used for:

- Continuous sample transmission
- High-speed streaming between DMA and RFDC

3.8 DMA Architecture

Direct Memory Access is responsible for transferring data without processor intervention.

Three DMA engines are used:

DMA	Function
AXI DMA 0	Transmit
AXI DMA 1	Receive I
AXI DMA 2	Receive Q

The transmit DMA continuously streams samples to the DAC.

The receive DMAs independently capture the I and Q channels from the ADC.

This architecture minimizes processor overhead and supports high-speed continuous streaming.

3.9 Packet Generator

A custom Packet Generator IP is included in the FPGA design.

Its primary functions are:

- Defining the number of samples to capture.
- Initiating receive operations.
- Synchronizing DMA transfers.
- Stopping data streaming after capture.

The packet generator allows controlled acquisition of finite data blocks for analysis.

3.10 Signal Transmission Path

The transmission process follows these steps:

1. A 20 MHz baseband sine wave is generated in software.
2. The waveform is stored in DDR memory.
3. AXI DMA transfers the waveform to the FPGA.
4. The FPGA streams the data to the RFDC.
5. The RFDC performs interpolation and digital upconversion.
6. The DAC converts the digital samples into an analog RF waveform.
7. The analog signal is available at the RF output connector.

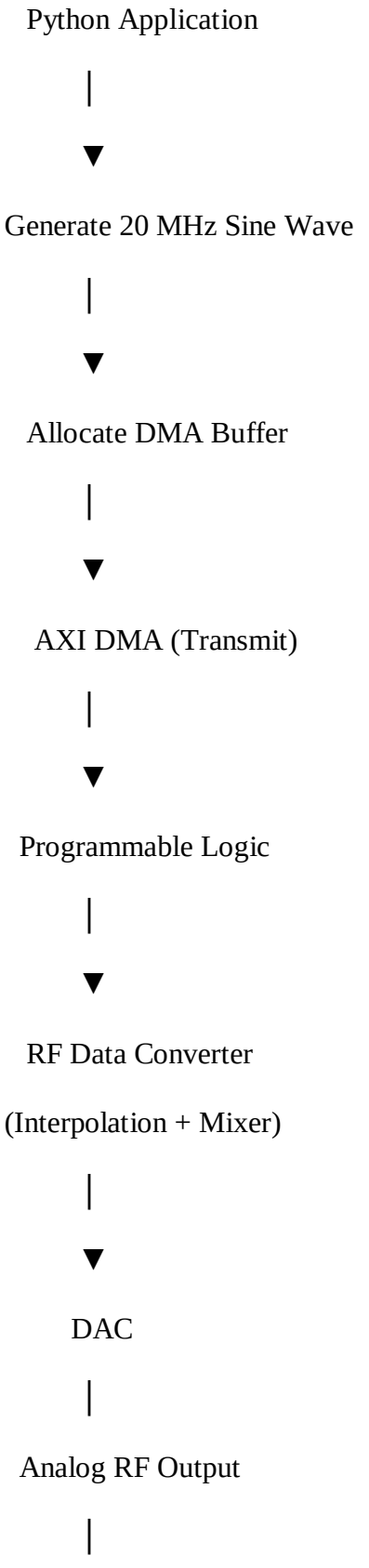
3.11 Signal Reception Path

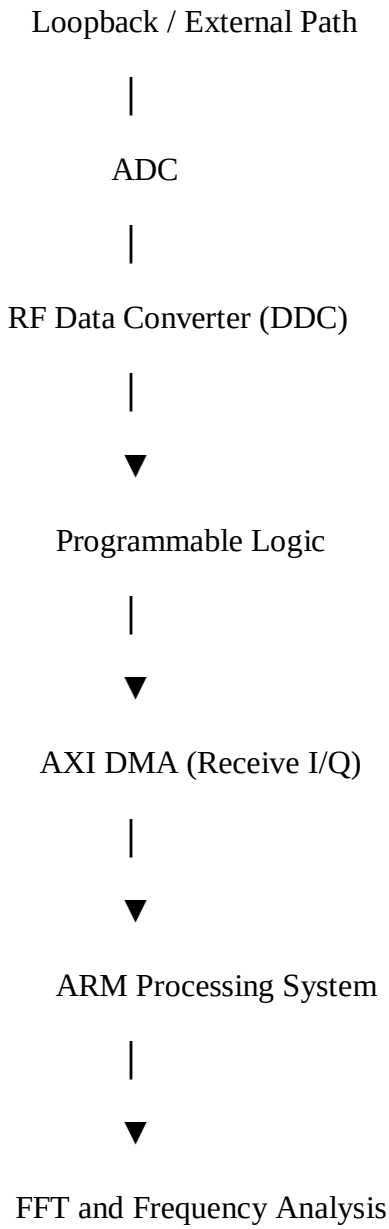
The reception process consists of the following stages:

1. The RF input is applied to the ADC.
2. The ADC digitizes the analog signal.
3. The RFDC performs digital downconversion.
4. The FPGA receives the digital samples.
5. AXI DMA transfers the I and Q samples to DDR memory.
6. The ARM processor reconstructs the complex baseband signal.
7. FFT analysis is performed to verify the received frequency.

3.12 Complete Data Flow

The overall data flow implemented in the project is shown below.





CHAPTER 4

SOFTWARE DESIGN AND IMPLEMENTATION

4.1 Introduction

The software component of the RFSoC transceiver was implemented using **Python** on the **PYNQ framework**. The PYNQ environment provides high-level APIs that enable software applications to configure and interact with FPGA hardware IP cores without requiring low-level driver development.

The software performs all control-oriented operations, while the FPGA executes the high-speed data path. The application configures the RF clocks, initializes the RF Data Converter (RFDC), generates the transmit waveform, manages Direct Memory Access (DMA) transfers, captures received samples, performs frequency-domain analysis, and visualizes the results.

The software follows a sequential execution flow while communicating with the FPGA through AXI interfaces.

4.2 Software Development Environment

The software was developed using the following tools.

Software	Purpose
Python 3	Application Development
PYNQ	Hardware Control
NumPy	Numerical Computing
SciPy	Signal Processing
Matplotlib	Data Visualization
Vivado	FPGA Design
RFSoC Overlay	Hardware Configuration

The FPGA hardware was first synthesized using Vivado. The generated bitstream (.bit) and hardware handoff (.hwh) files were then loaded by the Python application through the PYNQ Overlay class.

4.3 Software Architecture

The software is organized into several functional stages as illustrated below.

Initialization

Clock Configuration

RFDC Configuration

Signal Generation

DMA Transmission

Signal Reception

FFT Analysis

Visualization

4.4 Library Import and Environment Initialization

The first stage of the program imports the required software libraries.

The following categories of libraries are used.

Numerical Libraries

The NumPy library provides efficient multidimensional array operations required for waveform generation and mathematical computations.

SciPy provides signal processing functions including Fourier Transform and Power Spectral Density estimation.

Matplotlib is used for graphical visualization of both transmitted and received signals.

Hardware Libraries

The PYNQ framework provides interfaces to communicate with FPGA hardware.

The xrfclk package configures the RFSoC clock generators.

The xrfdc package configures the RF Data Converter.

The operating system environment variable

os.environ['BOARD']="RFSoC4x2"

ensures that the correct board drivers are loaded before accessing the hardware.

4.5 FPGA Overlay Loading

The FPGA hardware is initialized using

base = Overlay("design_1.bit")

The Overlay object performs the following operations.

- Programs the FPGA fabric.
- Loads the hardware description (.hwh).
- Detects all hardware IP blocks.
- Creates Python-accessible objects for each IP.

After initialization, the software gains access to:

- AXI DMA engines
- RF Data Converter
- Packet Generator
- AXI Interconnect

This eliminates the need for manually writing device drivers.

4.6 RF Clock Initialization

Reliable RF communication requires precise clock generation.

The RFSoC board contains two programmable clock generators.

- LMK
- LMX

The software initializes these clocks using

xrfclk.set_ref_clks()

The clock configuration used in the project is shown below.

Parameter	**Value**
LMK Frequency	245.76 MHz
LMX Frequency	491.52 MHz
ADC Sampling Rate	4915.2 MSPS
DAC Sampling Rate	4915.2 MSPS

Proper clock initialization ensures synchronized operation of both ADC and DAC.

4.7 RF Data Converter Configuration

The RFDC is configured before transmission begins.

The following operations are performed.

DAC Configuration

- Select DAC Tile 0.
- Configure DAC PLL.
- Set Nyquist Zone.
- Configure Mixer Frequency.
- Enable FIFO.
- Start DAC.

The DAC mixer frequency is configured to

$$300\ MHz$$

allowing digital upconversion of the generated baseband waveform.

ADC Configuration

Similarly,

the ADC configuration consists of

- Select ADC Tile 2.
- Configure PLL.
- Set Nyquist Zone.
- Configure Mixer Frequency.
- Enable FIFO.
- Start ADC.

The ADC mixer frequency is configured as

$$-300\ MHz$$

which performs digital downconversion

4.8 Baseband Signal Generation

The transmitter generates a baseband sinusoidal waveform.

The signal frequency is

$$20\ MHz$$

The mathematical expression is

$$x(t) = A\sin(2\pi f t)$$

where

$$A = 1$$

and

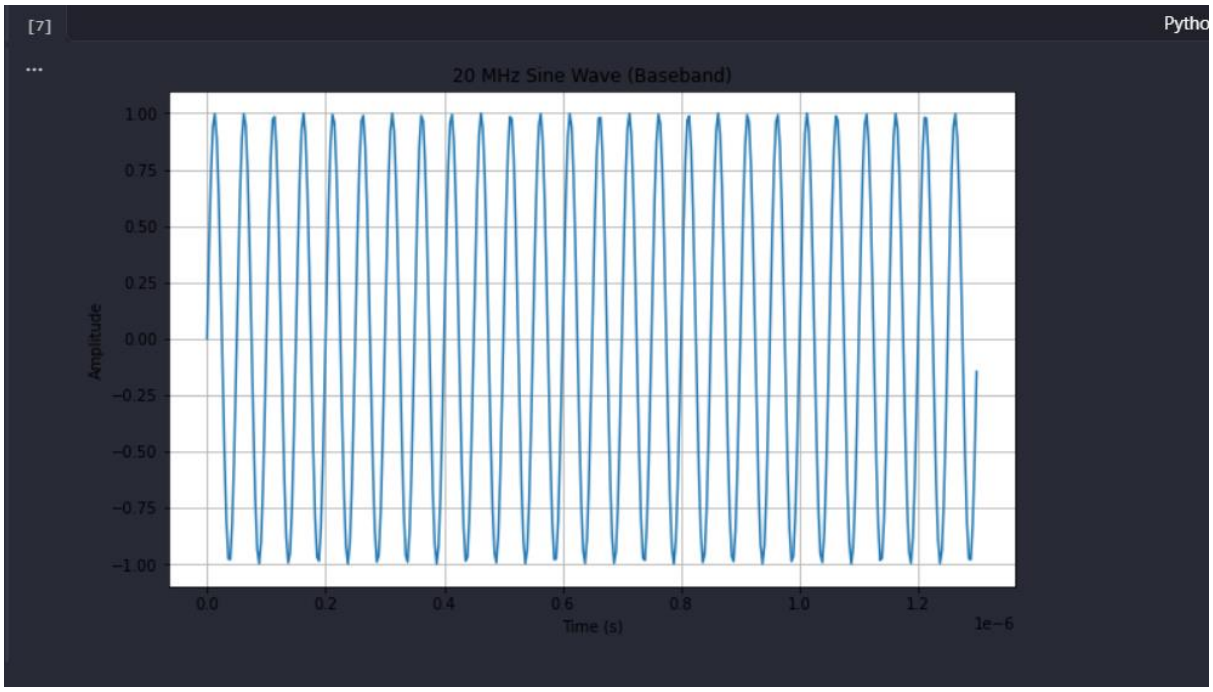
$$f = 20MHz$$

The effective baseband sampling frequency is

$$F_s = \frac{4.9152GHz}{16} = 307.2MHz$$

The waveform is generated using NumPy.

The generated signal represents the information to be transmitted through the RFSoC.



4.9 IQ Sample Formation

The RFSoC DAC accepts complex baseband samples.

Therefore,

the generated waveform is separated into

- In-phase (I)
- Quadrature (Q)

components.

Since the transmitted waveform is purely real,

$$Q = 0$$

The samples are interleaved as

$$[I_0, Q_0, I_1, Q_1, \cdots]$$

before transmission.

This data organization is required by the RFDC interface.

4.10 Signal Scaling

Before transmission,

the waveform amplitude is normalized.

The normalized samples are multiplied by

$$16384$$

to utilize the DAC input range without introducing clipping.

The scaling operation improves the Signal-to-Noise Ratio while maintaining linear operation of the DAC.

4.11 DMA Buffer Allocation

High-speed streaming requires physically contiguous memory.

The software allocates DMA buffers using

allocate()

instead of standard NumPy arrays.

The allocated buffer resides in DDR memory and is directly accessible by both the ARM processor and the FPGA.

The transmit buffer stores the interleaved IQ samples before transmission.

4.12 Transmit DMA

The transmission process is initiated using

sendchannel.transfer()

The transmit DMA continuously streams data from DDR memory into the FPGA.

The cyclic mode is enabled.

Consequently,

the waveform is transmitted repeatedly without CPU intervention.

The transmit path is

DDR

↓

AXI DMA

↓

FPGA

↓

RFDC

↓

DAC

↓

RF Output

4.13 Receive DMA

Two independent DMA engines capture

- I samples
- Q samples

from the RFDC.

Separate buffers are allocated for both channels.

The DMA engines automatically transfer the received samples into DDR memory.

The processor waits until both DMA transfers complete before proceeding.

4.14 Packet Generator

The Packet Generator controls the receive operation.

Its functions include

- defining the capture length,
- initiating data acquisition,
- synchronizing DMA transfers,
- stopping data capture.

Unlike communication packets,

this IP controls the timing of sample acquisition within the FPGA.

4.15 Complex Signal Reconstruction

The received I and Q samples are combined to reconstruct the complex baseband signal.

Mathematically,

$$r[n] = I[n] + jQ[n]$$

This complex representation enables efficient frequency-domain processing and supports future implementation of advanced modulation schemes.

4.16 Frequency Analysis

The received signal is transformed into the frequency domain using the Fast Fourier Transform.

The FFT is computed using

numpy.fft.fft()

The resulting spectrum is shifted using

fftshift()

so that the zero-frequency component appears at the center of the plot.

The frequency axis is computed according to

$$f = \frac{kF_s}{N}$$

where

$$N$$

is the FFT size.

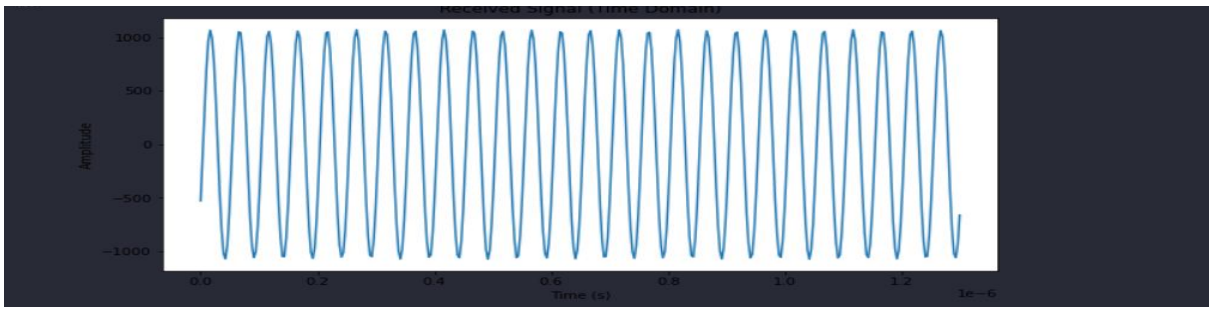
4.17 Visualization

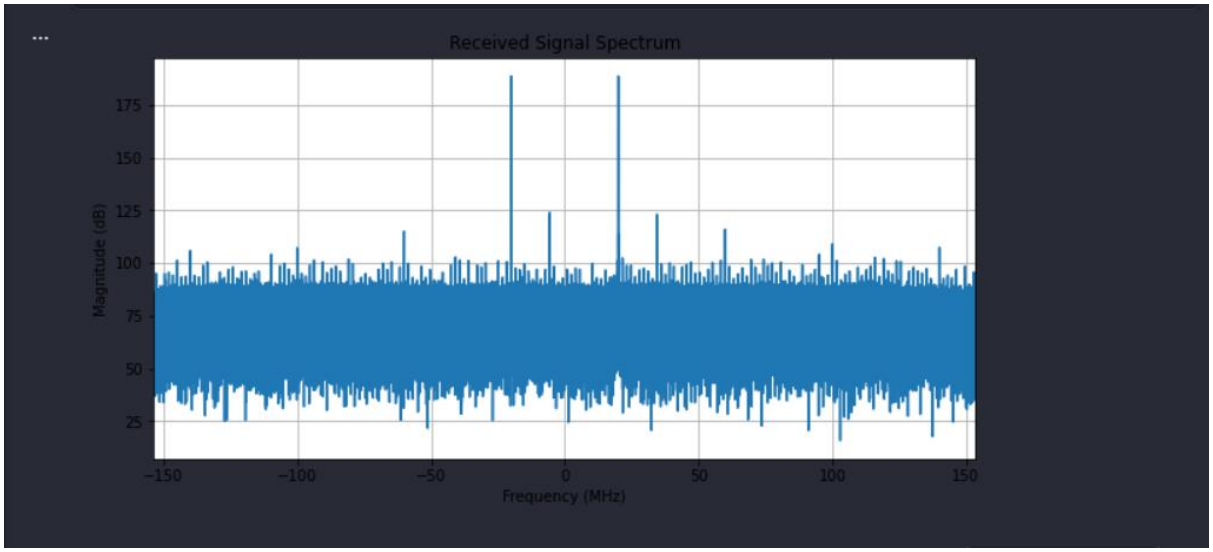
Two graphical outputs are generated.

Time-Domain Plot

Displays the received waveform.

This verifies correct waveform reconstruction.





Displays the FFT magnitude.

A dominant spectral peak appears near

$$20MHz$$

confirming successful transmission and reception.

[Insert Figure 4.8: FFT Spectrum of Received Signal]

4.18 Frequency Verification

The software automatically determines the dominant received frequency.

The FFT magnitude is searched for the maximum value.

The corresponding frequency bin is reported.

If the received frequency matches

$$20MHz$$

the transmitter–receiver chain is considered to operate correctly.

4.19 Complete Software Execution Flow

The complete execution sequence is summarized below.

Start

|

▼


Import Libraries
Load FPGA Overlay
Configure RF Clocks
Configure DAC
Configure ADC
Generate Sine Wave
IQ Interleaving
Allocate DMA Buffer
Transmit via DMA


Receive Samples
FFT
Display Results